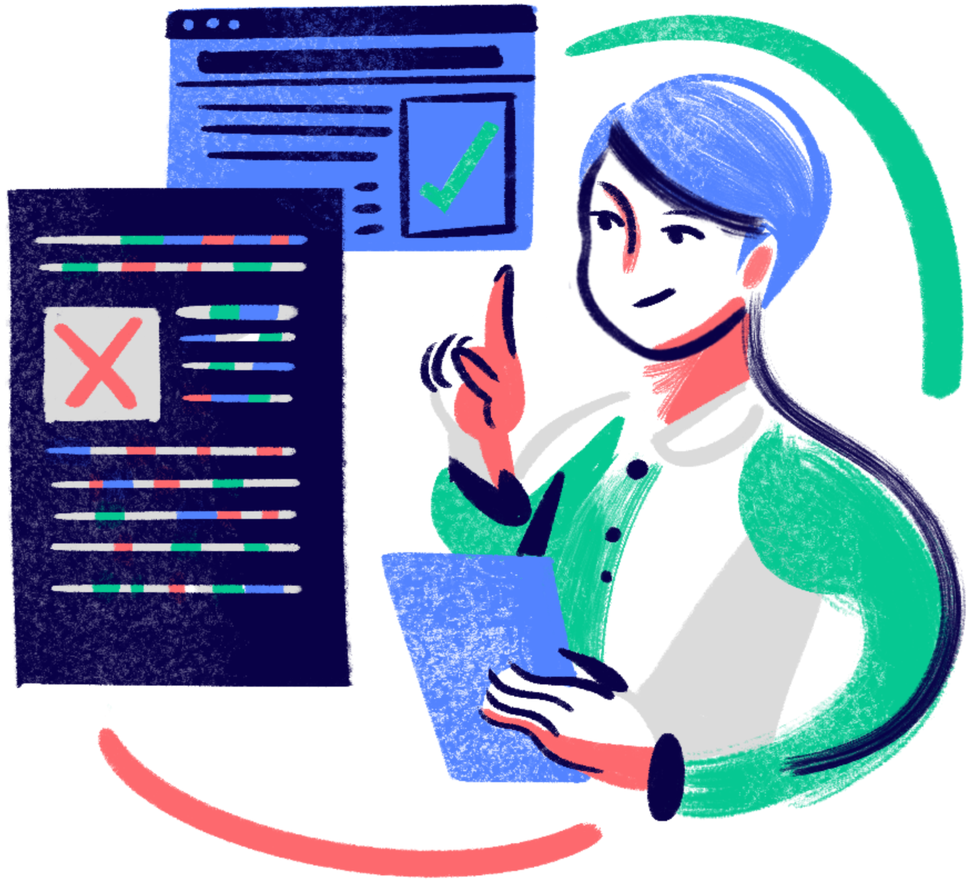
STQA

PANDA-LITE API

Sytem Testing



PRESENTED TO **MUBASWIRUL ISLAM**

PRESENTED BY **GROUP B2**

LOW

MEDIUM

MAJOR

CRITICAL

Orders Endpoint

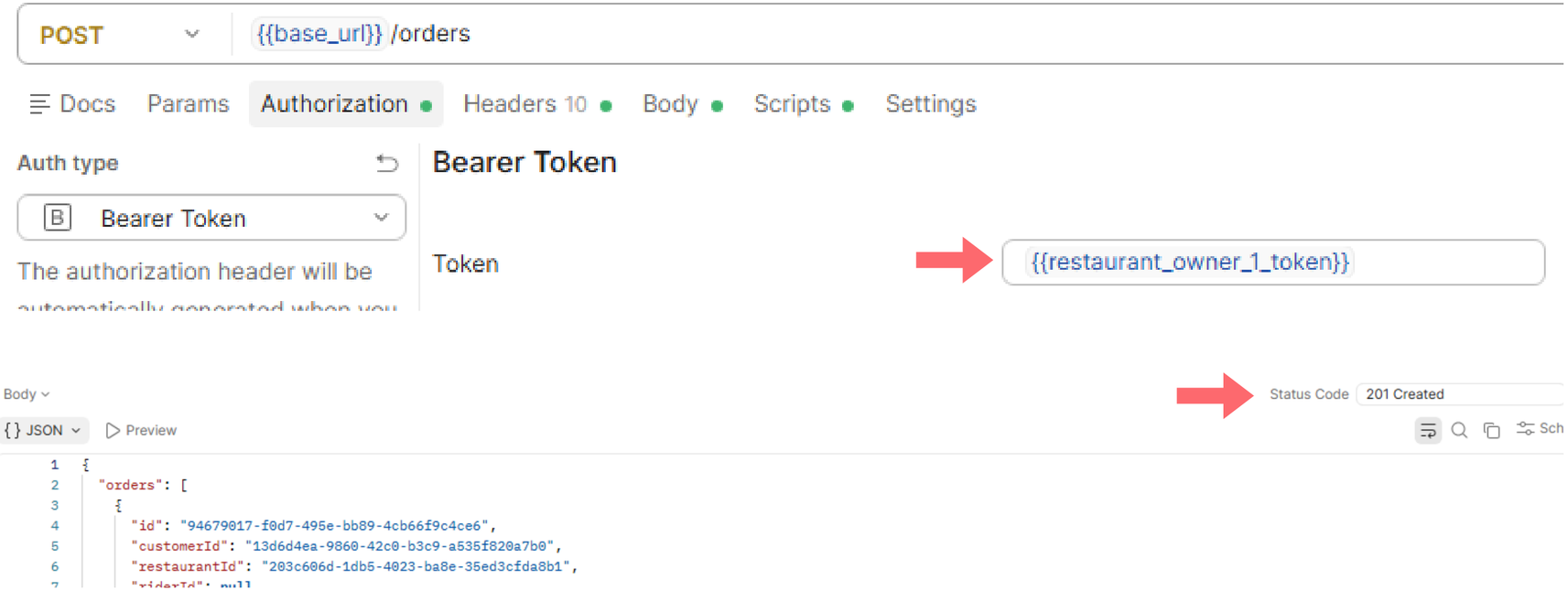
POST /orders

CRITICAL ID: 9cda25

01 RESTAURANT OWNER CAN PLACE CUSTOMER ORDER

A restaurant account can perform an operation reserved for customers and create an order.

EXPECTED:	ACTUAL:
403 - Forbidden	201 - Created



Steps to Reproduce:

1. Authenticate as Restaurant Owner 1.

2. Use a valid open restaurant and valid menu item with sufficient stock.

3. Send POST /orders using the restaurant owner's Bearer token.

4. Observe the status and created order.

Orders Endpoint

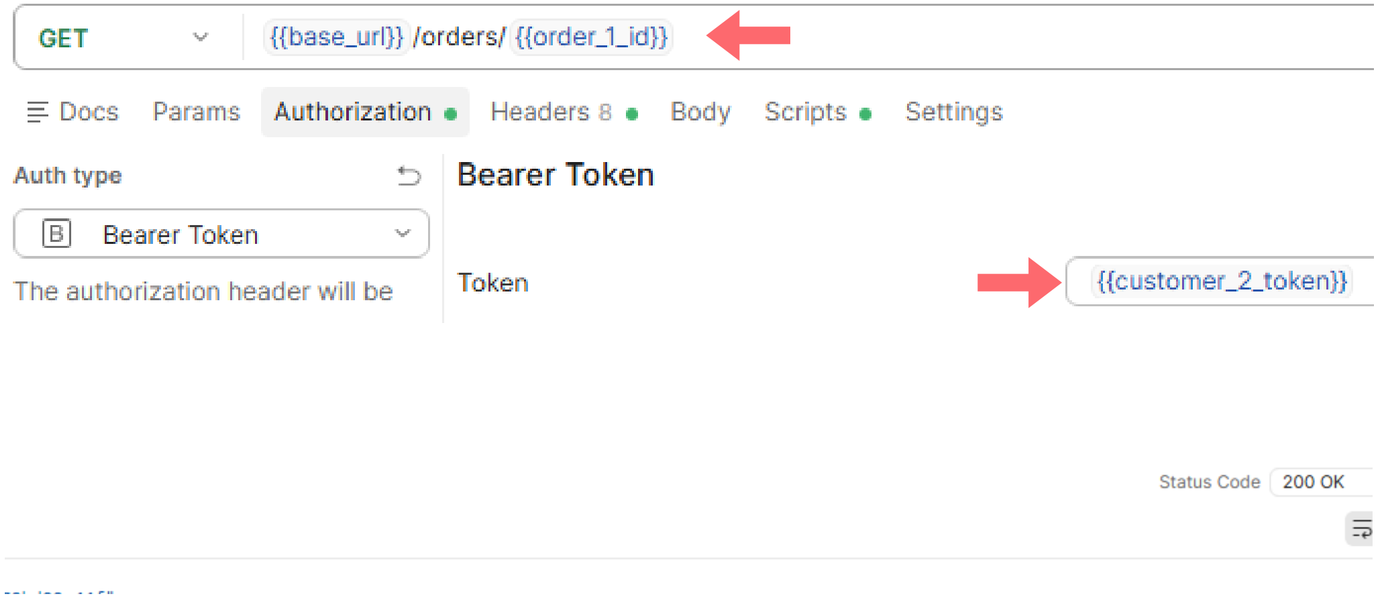
`GET /orders/:id`

CRITICAL ID: 59c2fb

02 UNRELATED AUTHENTICATED USERS CAN VIEW ANOTHER CUSTOMER'S ORDER

The documented ownership/visibility authorization rule is bypassed.

EXPECTED:	ACTUAL:
403 - Forbidden	200 - OK



Steps to Reproduce:

1. Create Order 1 using Customer 1 at Restaurant 1.

2. Authenticate as Customer 2, who did not place the order.

3. Send GET /orders/:id for Order 1 and observe the response.

4. Repeat using Restaurant Owner 2, who does not own Restaurant 1.

5. Observe that both unrelated users can retrieve the order.

Orders Endpoint

POST /orders

MAJOR ID: 14946d

03 PLATFORM FEE CALCULATED FROM INCORRECT BASE, CAUSING INCORRECT TOTAL

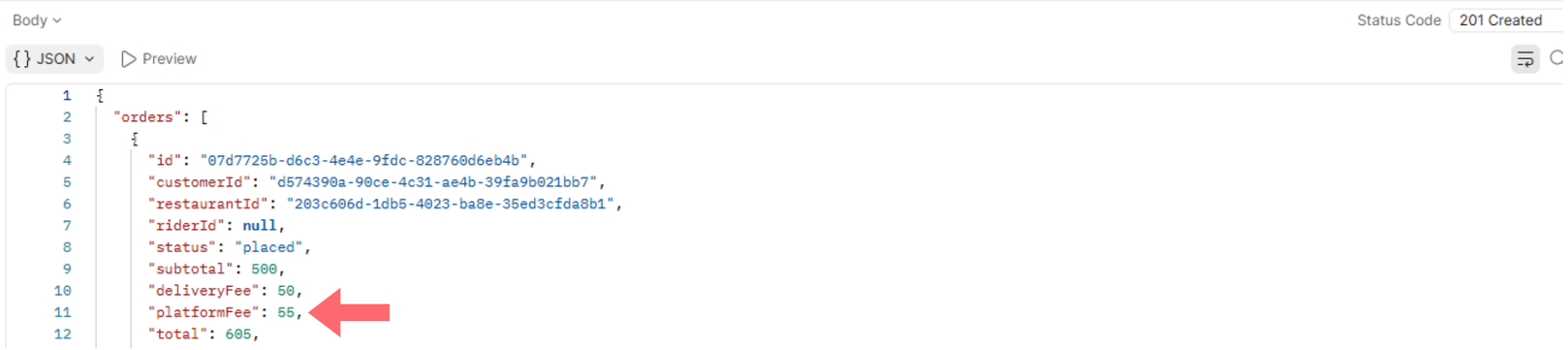
The documented platform fee is 10% of subtotal. The returned fee appears to be calculated using an incorrect base

EXPECTED:

201 - Created; subtotal = 500, delivery fee = 50, platform fee = 50 (10% of subtotal), total = 600

ACTUAL:

201 - Created; subtotal = 500, delivery fee = 50, platform fee = 55, total = 605



Steps to Reproduce:

1. Authenticate as Customer 1.

2. Use an open restaurant containing a menu item priced at 250 with sufficient stock.

3. Send POST /orders with quantity 2.

4. Observe subtotal, deliveryFee, platformFee, and total in the created order.

Orders Endpoint

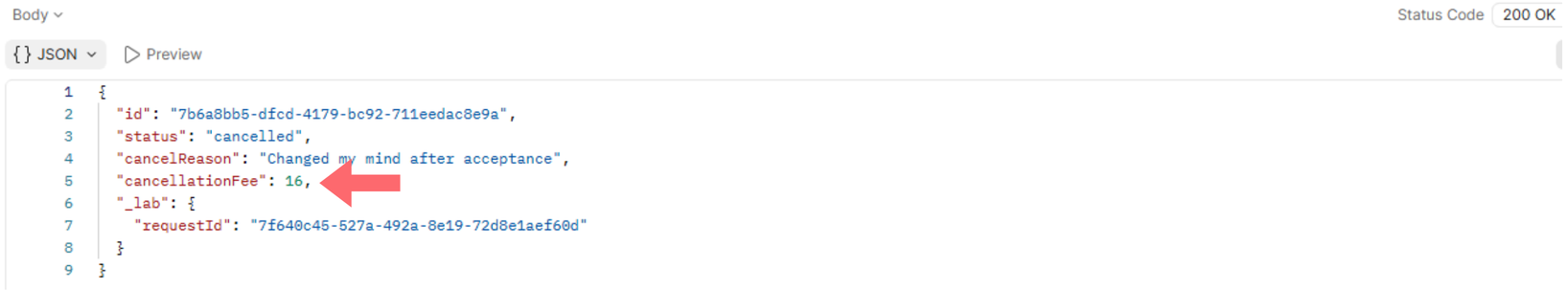
PATCH /orders/:id/cancel

MAJOR ID: aef60d

04 ACCEPTED ORDER USES INCORRECT CANCELLATION FEE

Cancellation from accepted should apply a penalty equal to the delivery fee. The configured delivery fee is 50, but the API reports 16

EXPECTED:	ACTUAL:
200 - OK; order becomes cancelled and cancellationFee = 50	200 - OK; order is cancelled but cancellationFee = 16



Steps to Reproduce:

1. Create an order as Customer 1.

2. Authenticate as the owning restaurant owner and move the order from placed to accepted.

3. Authenticate again as Customer 1.

4. Send PATCH /orders/:id/cancel with a cancellation reason.

5. Observe cancellationFee.

Orders Endpoint

PATCH /orders/:id/cancel

MAJOR ID: e1c570

05 PREPARING ORDER CAN STILL BE CANCELLED

The documented lifecycle prevents cancellation once preparation begins.

EXPECTED: 400 - Bad Request

ACTUAL: 200 - OK


```
{
  "id": "1fbd1e29-6a35-436b-975b-e68314b63045",
  "customerId": "fa9f019a-e5e7-40c5-981b-6442551e60f9",
  "restaurantId": "551315e2-ec36-4654-bfb3-f73cd2566b9c",
  "riderId": null,
  "status": "preparing",
```




Steps to Reproduce:

1. Create an order as Customer 1.
2. Move the order from placed to accepted using the owning restaurant owner's token.
3. Move it from accepted to preparing.
4. Authenticate as Customer 1.
5. Send PATCH /orders/:id/cancel.
6. Observe the response.

Orders Endpoint

POST /orders

MEDIUM

ID: 469849

06 NEGATIVE ORDER QUANTITY CAUSES INTERNAL SERVER ERROR

Invalid client input causes an unhandled server-side failure instead of a controlled validation response.

EXPECTED:

400 - Bad Request

ACTUAL:

500 - Internal Server Error

```
{
  "restaurantId": "{{restaurant_1_id}}",
  "items": [
    {
      "menuItemId": "{{edge_negative_quantity_item_id}}",
      "quantity": -1
    }
  ]
}
```

Status Code 500 Internal Server Error

ON Preview

```
{
  "error": "Internal server error",
  "_lab": {
    "requestId": "feee70ac-b7ae-4c57-b8dc-0adfb0469849"
  }
}
```

Steps to Reproduce:

1. Authenticate as Customer 1.

2. Create/use a valid menu item with positive stock.

3. Send POST /orders using the valid restaurant and menu item but set quantity to -1.

4. Observe the response status and body.

Orders Endpoint

POST /orders

LOW ID: aa9283

07 MISSING RESTAURANT ID RETURNS INCORRECT ERROR MESSAGE

The response does not match the documented validation message for a missing restaurantId.

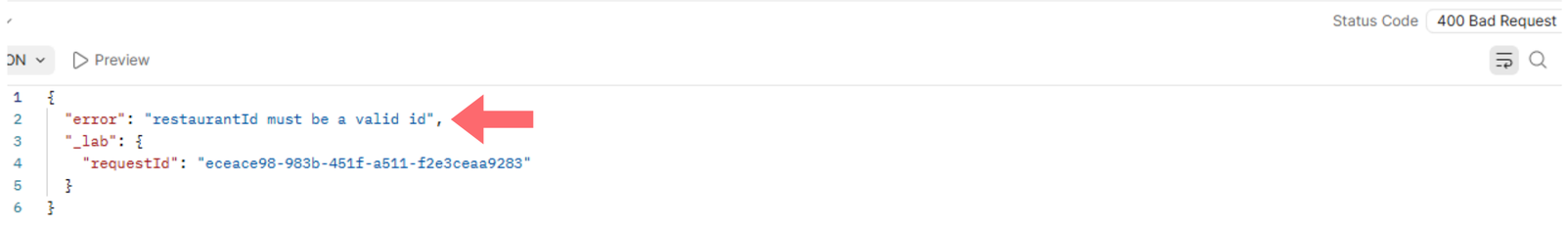
EXPECTED:

400 - Bad Request with "restaurantId and at least one item are required"

ACTUAL:

400 - Bad Request with "restaurantId must be a valid id"

```
{
  "items": [
    {
      "menuItemId": "{{order_negative_menu_item_id}}",
      "quantity": 1
    }
  ]
}
```



Steps to Reproduce:

1. Authenticate as Customer 1.

2. Send POST /orders with a valid items array but omit restaurantId.

3. Observe the status code and error body.

Auth & Registration — Defect Findings

Area: Registration, Login

Major

Password char 5 accepted

Defect ID: a829f9

A 5-character password is accepted and the account is created, even though the documented minimum is 6 characters.

EXPECTED:

400 Bad Request

ACTUAL:

201 Created



Steps to Reproduce:

1. Send POST /auth/register with valid name, unique email, valid role, and a 5-character password
2. Observe the response.



Auth & Registration — Defect Findings

Area: Registration, Login

Minor

Empty password triggers the wrong validation message

Defect ID: e79dc3

An empty password correctly returns 400, but triggers the generic "missing fields" message instead of the password-length message.

EXPECTED:

400 Bad Request "Password must be at least 6 characters long"

ACTUAL:

400 Bad Request "name, email, password and role are required"

Status Code 400 Bad Request

Preview


```
{
  "error": "name, email, password and role are required",
  "_lab": {
    "requestId": "a1ca1dd7-dc5e-46f1-92f2-e76383e79dc3"
  }
}
```


Steps to Reproduce:

1. Send POST /auth/register with valid name, unique email, valid role, and an empty password field.
2. Observe status code and error message.



GET /restaurants — Defects at a Glance

List endpoint — filtering and sorting query params

MEDIUM

01 CREATEDAT NEVER MATCHES ID: b7619b

Exact, valid createdAt timestamp returns an empty array instead of the matching restaurant.

EXPECTED:

200-OK
restaurant returned

ACTUAL:

200-OK
empty array



Steps to Reproduce:

1. Copy the exact createdAt timestamp from an existing restaurant record.
2. Send GET /restaurants?filter_field=createdAt&filter_value=<timestamp> (tested raw and URL-encoded).
3. Observe response body.

GET /restaurants — Defects at a Glance

List endpoint — filtering and sorting query params

MEDIUM

02 SORT_BY= (EMPTY) SILENTLY DEFAULTS ID: a366899

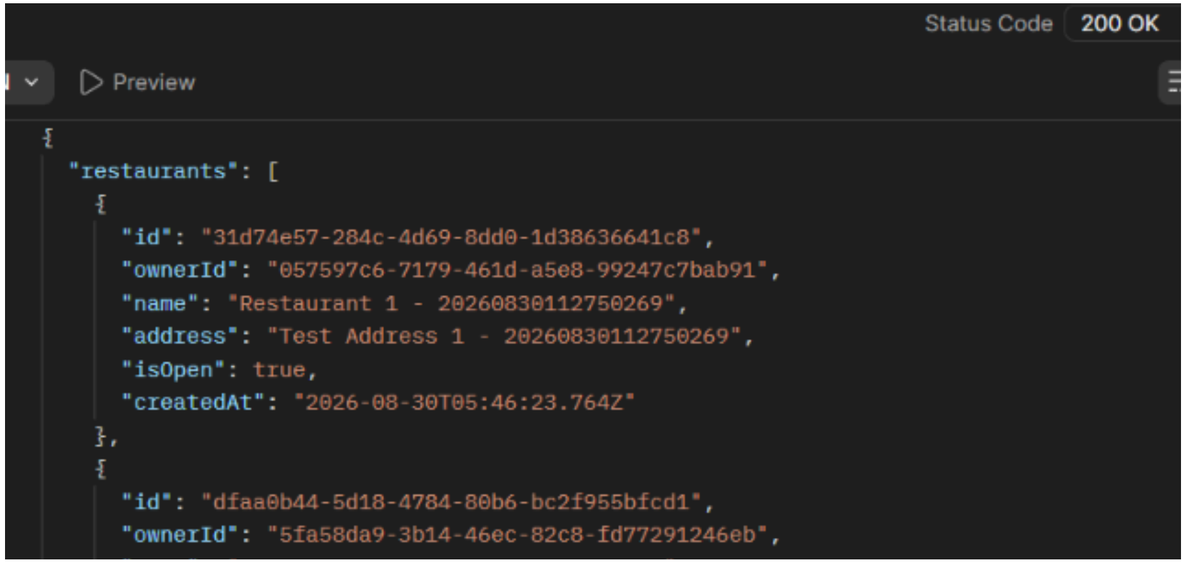
Empty sort_by should be rejected (400) but is accepted and falls back to the default sort.

EXPECTED:

400-BAD Request

ACTUAL:

200-OK



Steps to Reproduce:

1. Send GET /restaurants?sort_by= (empty value).
2. Observe status code and sort order.

GET /restaurants — Defects at a Glance

List endpoint — filtering and sorting query params

LOW MEDIUM

03 DUPLICATE SORT_BY PARAMS ACCEPTED ID: 95079c

sort_by=asc&sort_by=desc returns 200 with no documented precedence rule.

EXPECTED:

400-BAD Request

ACTUAL:

200-OK



Steps to Reproduce:

1. Send GET /restaurants?sort_by=asc&sort_by=desc.

2. Observe status code and which value wins.

GET /restaurants — Defects at a Glance

List endpoint — filtering and sorting query params

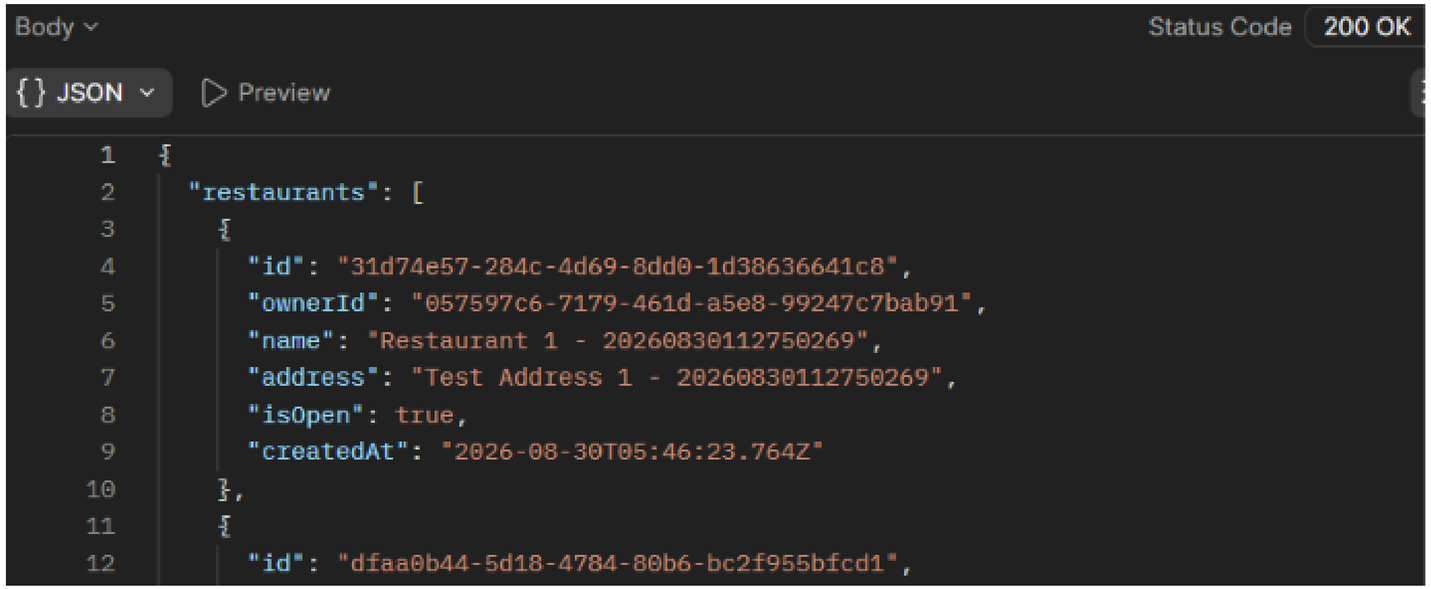
LOW MEDIUM

04 SORT_BY=ASC (UPPERCASE)

ID: e743d8

sort_by=ASC(UPPERCASE) returns 200 ,instead of 400-Bad Request

EXPECTED:	ACTUAL:
400-BAD Request	200-OK accepted and sorted correctly (ascending).



Steps to Reproduce:

1. Send GET /restaurants?sort_by=ASC (uppercase).
2. Observe status code and sort order.

/restaurants/:id — Defects at a Glance

GET and PATCH by id — id validation, case sensitivity, and ownership

01

RESTAURANT NOT FOUND(MALFORMED/NON-UUID ID)

ID: b55813

EXPECTED:

404-Not Found

ACTUAL:

500-Internal Server Error



Steps to Reproduce:

1. Send GET /restaurants/12345 (non-UUID-format id).

2. Observe status code.

Status Code 500 Internal Server Error

Preview


```
{
  "error": "Internal server error",
  "_lab": {
    "requestId": "f7c26c6a-581e-4b22-9174-eea81426fa01"
  }
}
```

/restaurants/:id — Defects at a Glance

GET and PATCH by id — id validation, case sensitivity, and ownership

02

RESTAURANT NOT FOUND(NUMERIC-ONLY ID)

ID: 26fa01

EXPECTED:

404-Not Found

ACTUAL:

500-Internal Server Error



Steps to Reproduce:

1. Send GET /restaurants/123456789 (numeric-only, non-UUID id).

2. Observe status code.

Body — Status Code: 500 Internal Server Error

{} JSON — Preview


```
{
  "error": "Internal server error",
  "_lab": {
    "requestId": "f7c26c6a-581e-4b22-9174-eea81426fa01"
  }
}
```

/restaurants/:id — Defects at a Glance

GET and PATCH by id — id validation, case sensitivity, and ownership

03 EDGE CASE

RESTAURANT NOT FOUND(INJECTION-STYLE ID)

ID: ae81e7

EXPECTED:

404-Not Found

ACTUAL:

500-Internal Server Error



Steps to Reproduce:

1. Send GET /restaurants/' OR '1'='1 (SQL-injection-style id).

2. Observe status code and response body.

Status Code 500 Internal Server Error

Preview


```
{
  "error": "Internal server error",
  "_lab": {
    "requestId": "f7c26c6a-581e-4b22-9174-eea81426fa01"
  }
}
```

/restaurants/:id — Defects at a Glance

GET and PATCH by id — id validation, case sensitivity, and ownership

04 Medium

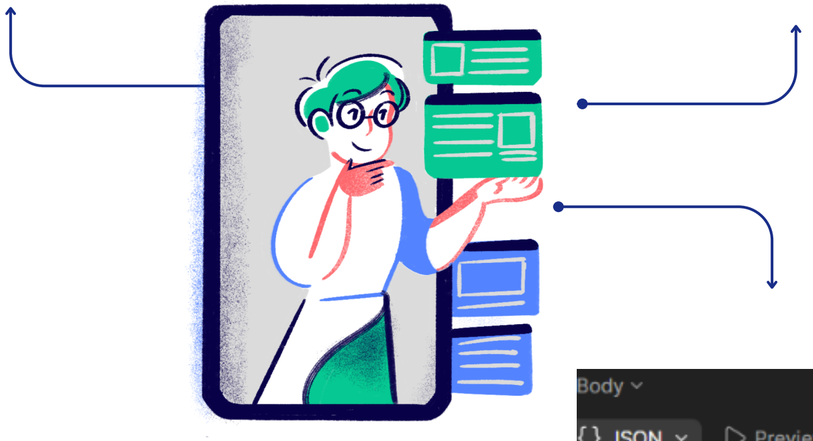
RESTAURANT NOT FOUND(CASE SENSITIVITY-UPPERCASE) ID: e9646b

EXPECTED:

404-Not Found

ACTUAL:

200-OK



Steps to Reproduce:

1. Take an existing restaurant's valid UUID and uppercase it.
2. Send GET /restaurants/<UPPERCASED_UUID>.
3. Observe status code.

Body — Status Code 200 OK

{} JSON — Preview


```
{
  "id": "3dc3996a-7955-4e6e-96d5-3d4b391210a4",
  "ownerId": "c45fc086-54ea-419d-bffe-588667511e6e",
  "name": "Restaurant 4 - 20260830112750269",
  "address": "Test Address 4 - 20260830112750269",
  "isOpen": true,
  "createdAt": "2026-08-30T05:54:09.600Z",
  "_lab": {
    "requestId": "f475e1cf-f2b2-46f0-a143-4234c4e9646b"
  }
}
```

Ownership check is not enforced on PATCH

Defect ID: d46f2a / 34306b

Any authenticated restaurant owner can modify a restaurant they do not own — name, address, and open/closed status can all be overwritten by an unrelated account.

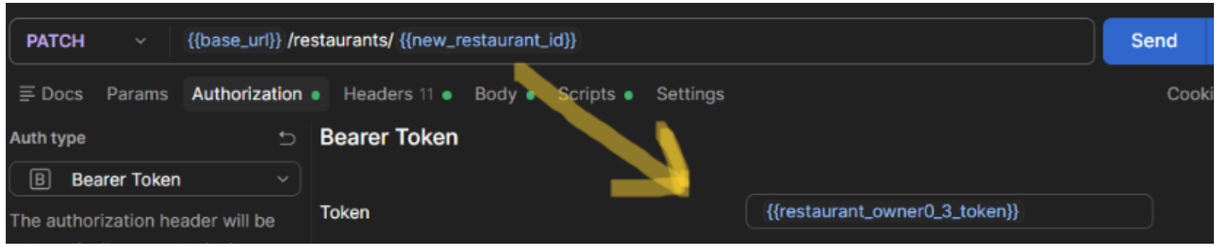
Critical

Token used:

restaurant_owner0_3_token

Target:

new_restaurant (owned by another user)



EXPECTED: **403-Forbidden**

"Only the restaurant owner can update this restaurant"

ACTUAL: **200 OK**

Both reproductions succeeded — the restaurant's data was overwritten by a non-owner.

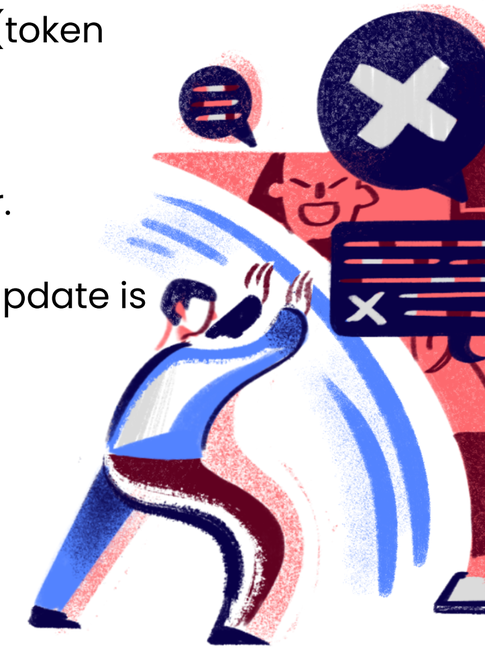


Steps to Reproduce:

1. Authenticate as restaurant_owner0_3 (token for a restaurant they do not own)

2. Send PATCH /restaurants/:id targeting new_restaurant, owned by a different user.

3. Observe status code and whether the update is applied.



Menu Items

PATCH /menu-items

MAJOR ID: 4f47eb

01 UPDATING MENU STOCK TO ZERO DOES NOT DISABLE ITEM

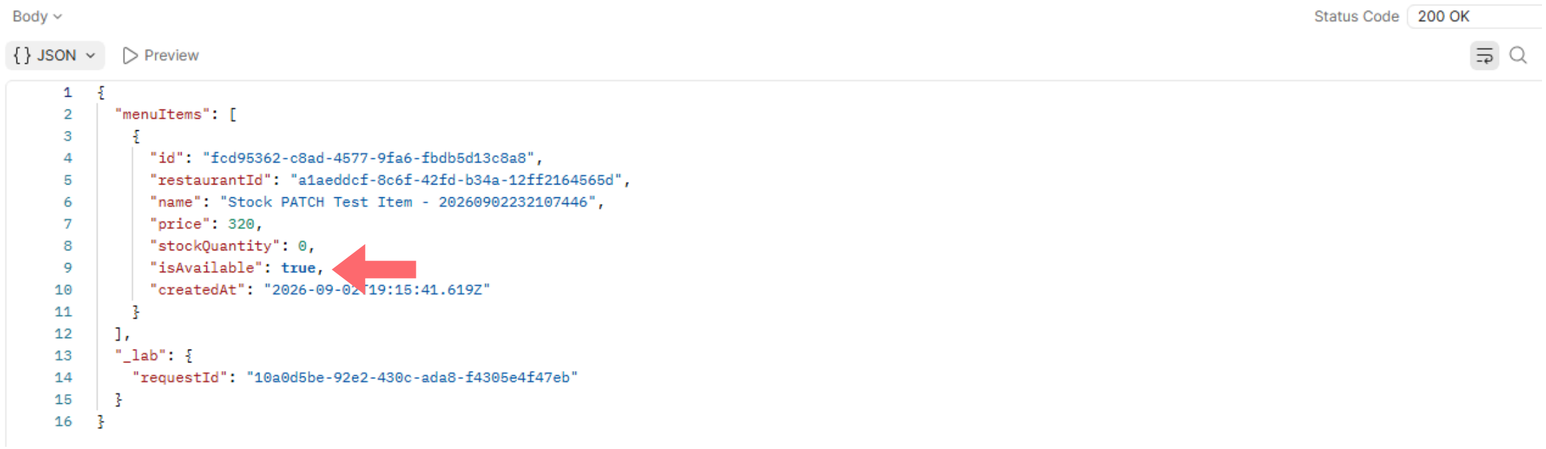
The stock changes correctly, but availability remains inconsistent with the stock state.

EXPECTED:

200 - OK, stockQuantity=0 and isAvailable=false

ACTUAL:

200 - OK, stockQuantity=0 but isAvailable=true



Steps to Reproduce:

1. Authenticate as the restaurant owner.

2. Use an existing menu item with positive stock and isAvailable=true.

3. Send PATCH /menu-items/:id with {"stockQuantity":0}.

4. Observe the returned menu item.

Menu Items

`PATCH /menu-items/:id`

MAJOR ID: 36062b

02 NEGATIVE PRICE CAUSES INTERNAL SERVER ERROR

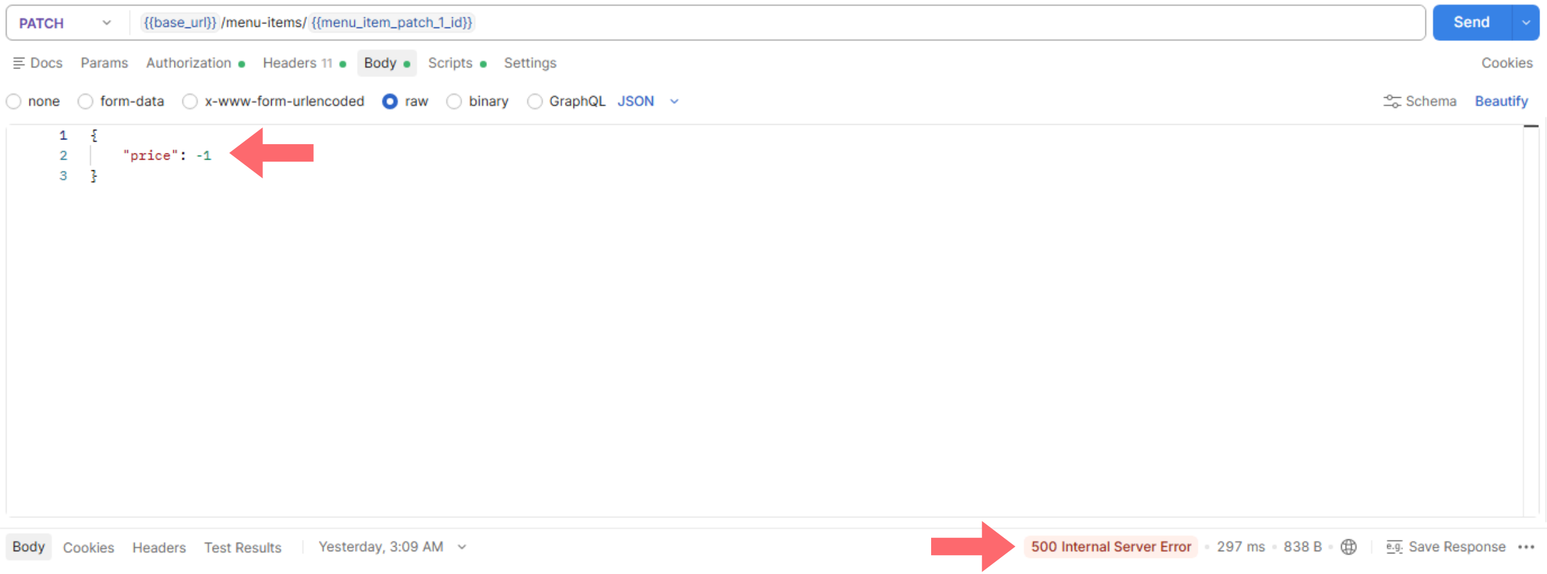
Invalid input reaches an unhandled server-side failure

EXPECTED:

400 - Bad Request

ACTUAL:

500 - Internal Server Error



Steps to Reproduce:

1. Authenticate as the restaurant owner of an existing menu item.

2. Send PATCH /menu-items/:id with {"price":-1}.

3. Observe the response status and error body.

Menu Items

PATCH /menu-items/:id

MAJOR ID: 352812

03 NEGATIVE STOCK QUANTITY CAUSES INTERNAL SERVER ERROR

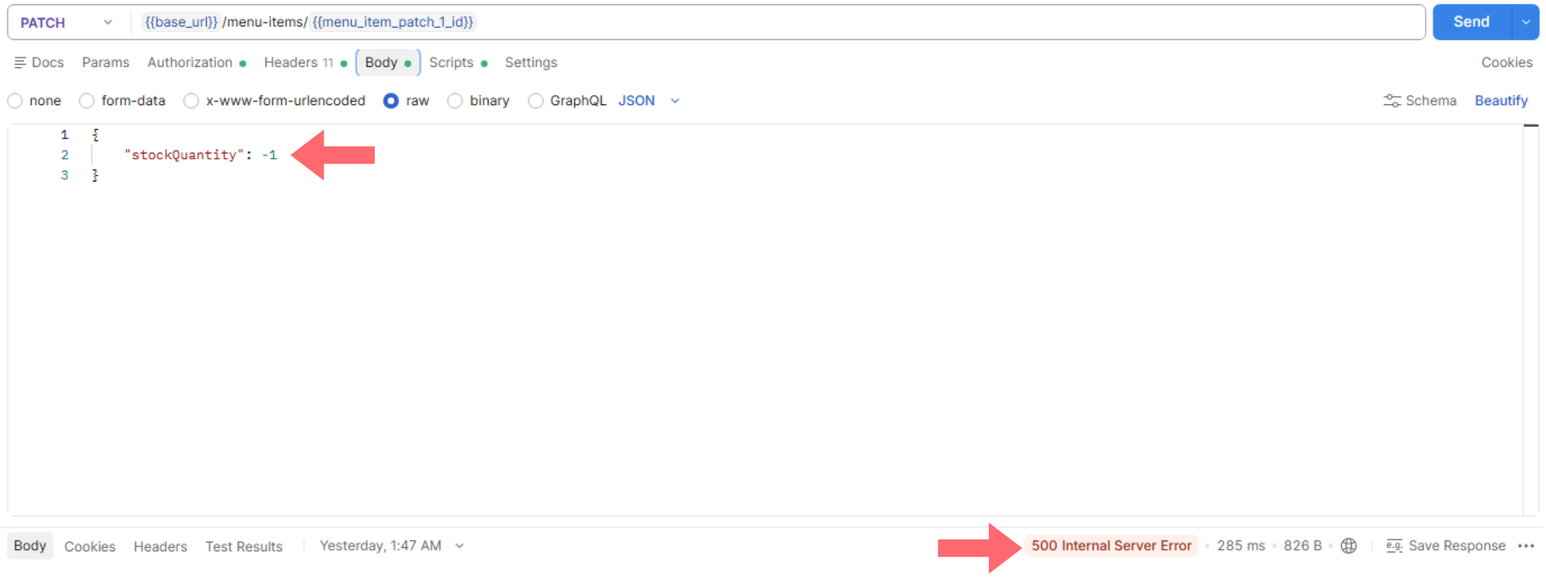
The stock changes correctly, but availability remains inconsistent with the stock state.

EXPECTED:

400 - Bad Request

ACTUAL:

500 - Internal Server Error



Steps to Reproduce:

1. Authenticate as the restaurant owner of an existing menu item.

2. Send PATCH /menu-items/:id with a negative stockQuantity value.

3. Observe the response status and error body.

Menu Items

POST /orders/:id/rate

MAJOR

ID: 62ab13

04 RATING - SCORE BELOW ALLOWED RANGE CAUSES INTERNAL SERVER ERROR

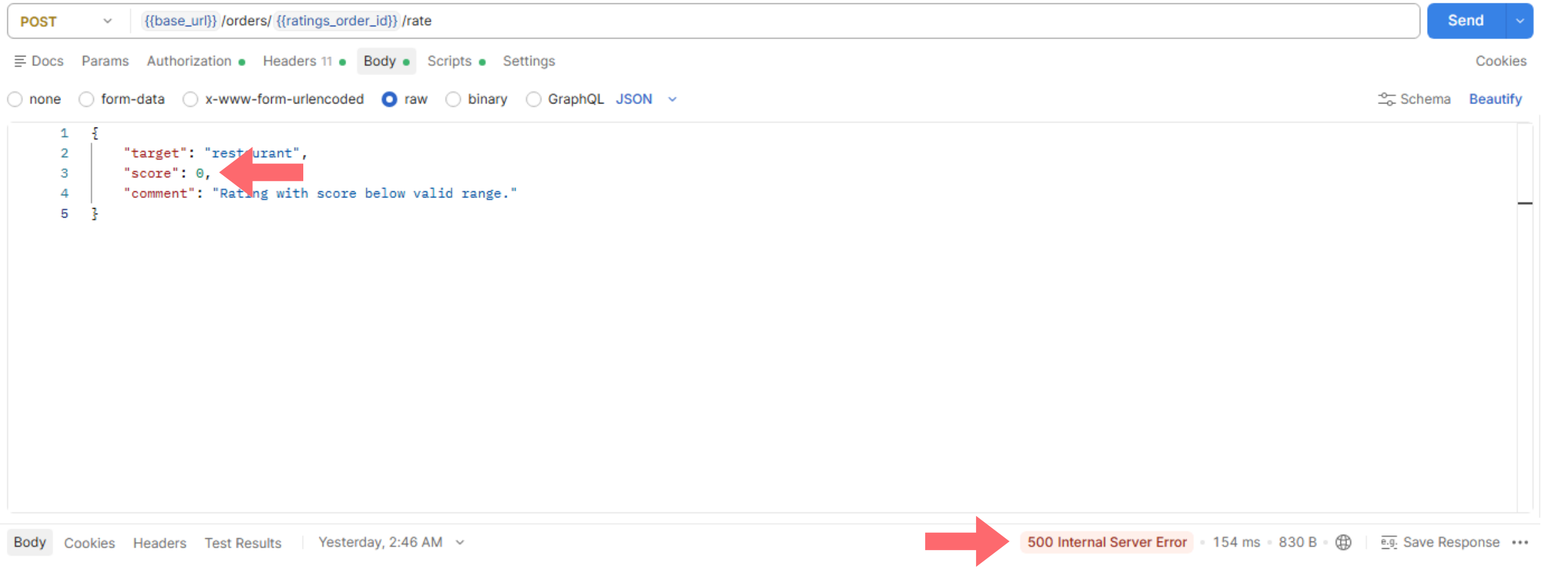
A below-range value should produce the documented validation error

EXPECTED:

400 - Bad Request

ACTUAL:

500 - Internal Server Error



Steps to Reproduce:

1. Use a delivered order belonging to the authenticated customer.

2. Send POST /orders/:id/rate with a valid target and score=0.

3. Observe the response.

Menu Items

POST /orders/:id/rate

MAJOR

ID: 976c38

05 RATING – SCORE ABOVE ALLOWED RANGE CAUSES INTERNAL SERVER ERROR

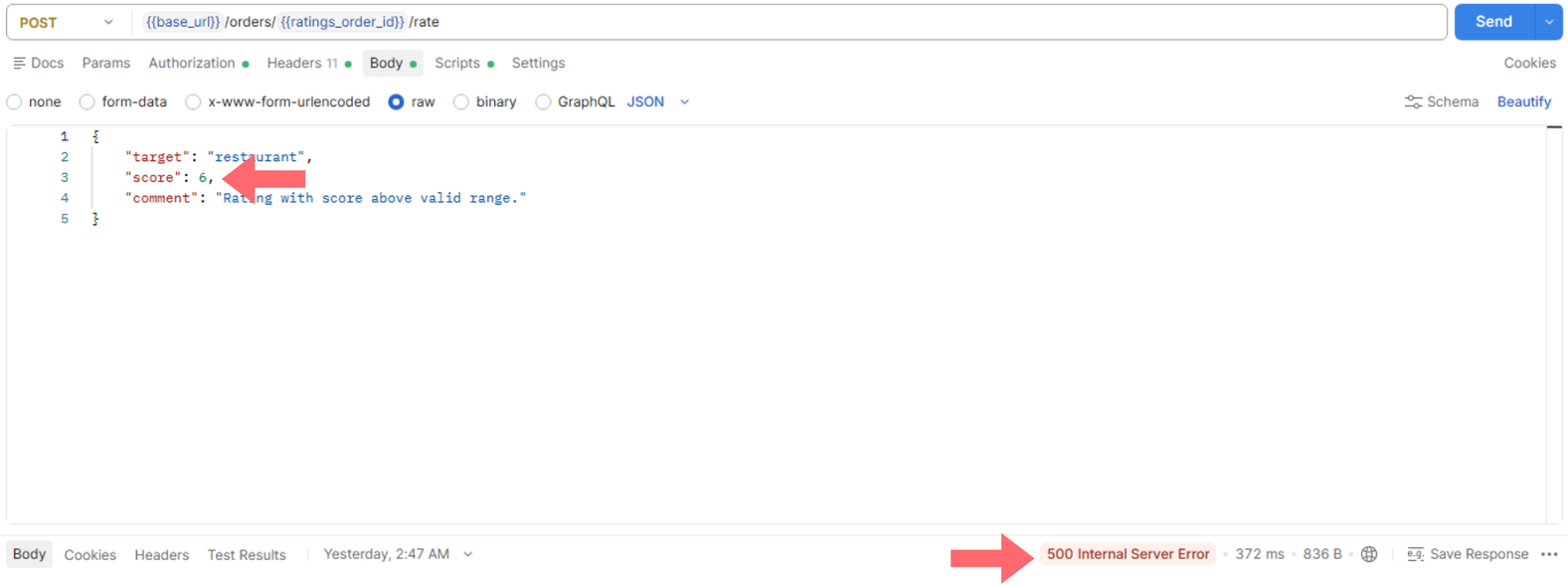
An above-range value should produce the documented validation error

EXPECTED:

400 - Bad Request

ACTUAL:

500 - Internal Server Error



Steps to Reproduce:

1. Use a delivered order belonging to the authenticated customer.

2. Send POST /orders/:id/rate with a valid target and score=6.

3. Observe the response.

Menu Items

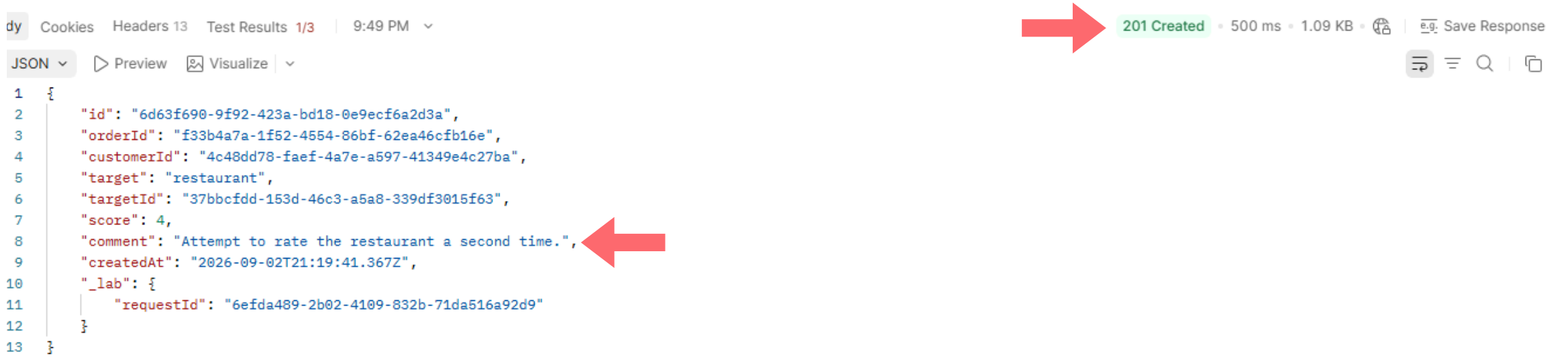
POST /orders/:id/rate

MAJOR ID: 9d47c6

06 DUPLICATE RESTAURANT RATING IS ACCEPTED

The API should allow only one rating per target for an order.

EXPECTED:	ACTUAL:
409 - Conflict	201 - Created



Steps to Reproduce:

1. Deliver an order belonging to the authenticated customer.

2. Successfully submit a restaurant rating for that order.

3. Submit another valid rating with target=restaurant for the same order.

4. Observe the response and created rating.

Menu Items

POST /orders/:id/rate

MEDIUM

ID: c3a6d8

07 MENU LIST - INVALID BOOLEAN FILTER VALUE ACCEPTED

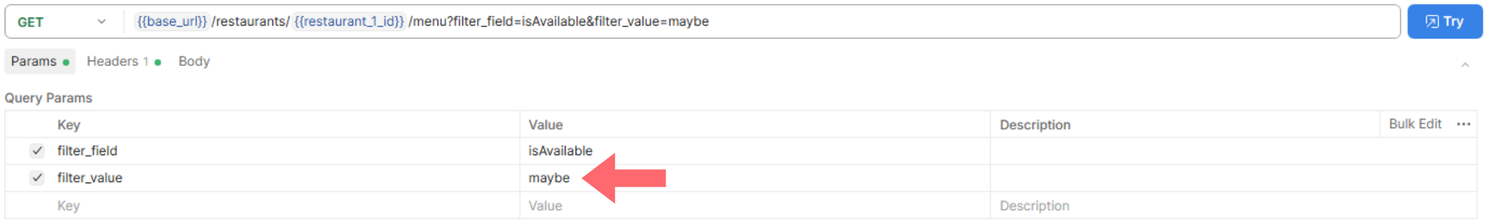
isAvailable is a boolean field, but an invalid boolean value such as maybe is accepted

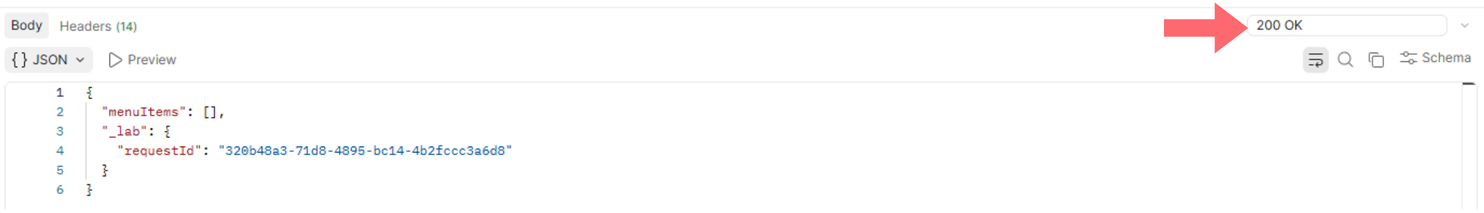
EXPECTED:

400 - Bad Request

ACTUAL:

200 - OK





Steps to Reproduce:

1. Authenticate with a valid token.

2. Send GET /restaurants/:id/menu?filter_field=isAvailable&filter_value=maybe for an existing restaurant.

3. Observe the status code and response body.

Order Lifecycle & Riders

`GET /orders/:id/timeline`

CRITICAL ID: 55ecee

01 UNAUTHORIZED RIDER CAN VIEW ANOTHER RIDER'S ORDER TIMELINE

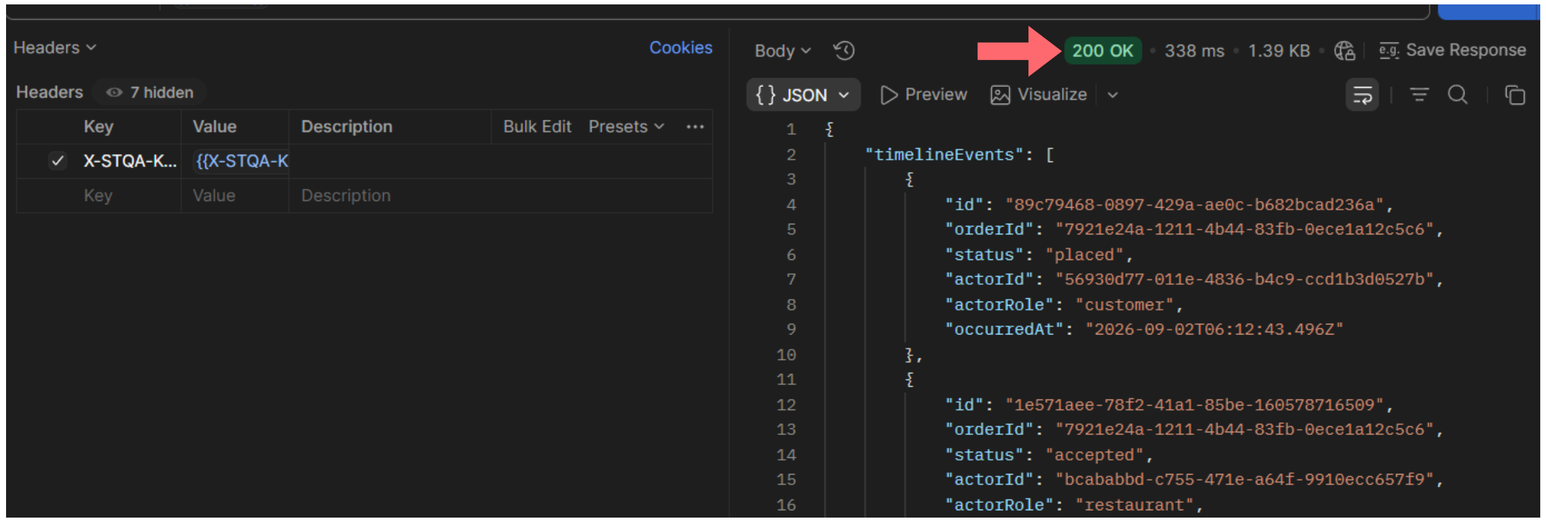
Authorization check is bypassed for timeline access. An unrelated rider can read another order's timeline

EXPECTED:

403 - Forbidden

ACTUAL:

200 - OK



Steps to Reproduce:

1. Create an order and progress it until it is assigned to Rider 1.

2. Authenticate separately as Rider 2, who is not the customer, owning restaurant owner, or assigned rider of the order.

3. Send GET /orders/:id/timeline using Rider 2's token.

4. Observe the status code and returned timeline.

Thank You!

